

Financial Fraud Detection using SQL

BY - MANISHA RAI

From Raw Data to Powerful Insights 

1. Introduction

This project demonstrates how SQL can be used to analyze financial transactions and detect fraud patterns.

Using real-world-like transaction data, we extract insights such as suspicious transactions, high-value frauds, and customer behavior patterns.



2. Raw Dataset (Transactions Table)

Table: transactions

Column	Description
txn_id	Transaction ID
customer_id	Customer ID
customer_name	Name
merchant	Merchant name
amount	Transaction amount
txn_type	Online / Offline
city	Transaction city
status	Success / Fraud

3. Creating Table (SQL Syntax)

```
CREATE TABLE transactions (  
    txn_id INT PRIMARY KEY,  
    customer_id INT,  
    customer_name VARCHAR(100),  
    merchant VARCHAR(100),  
    amount INT,  
    txn_type VARCHAR(50),  
    city VARCHAR(50),  
    status VARCHAR(20)  
);
```

[illegible]



4. Highest Transaction Detection

◆ SQL Query:

```
SELECT customer_name, amount  
FROM transactions  
WHERE amount = (SELECT MAX(amount) FROM transactions);
```



Output:

- Customer with highest transaction amount is identified

	customer_name	amount
▶	Vikas Rao	200000
	Amit Sharma	200000



Insight:

High-value transactions often indicate potential fraud risk.

5. Fraud Transaction Analysis

◆ SQL Query:

```
SELECT customer_name, amount  
FROM transactions  
WHERE status = 'Fraud';
```

Output:

- List of all fraud transactions

customer_name	amount
Ravi Kumar	50000
Arjun Mehta	90000
Karan Patel	150000
Vikas Rao	200000
Rahul Jain	60000
Amit Sharma	200000

Insight:

Fraud transactions are mostly high-value and irregular in pattern.



6. Above Average Spending Customers

◆ SQL Query:

```
SELECT customer_name, amount
FROM transactions
WHERE amount > (SELECT AVG(amount) FROM transactions);
```



Output:

- Customers spending above average

customer_name	amount
Arjun Mehta	90000
Karan Patel	150000
Vikas Rao	200000
Rahul Jain	60000
Amit Sharma	200000



Insight:

Unusual spending behavior helps identify risk users.



7. Same Merchant Pattern Analysis

◆ SQL Query:

```
SELECT customer_name, merchant
FROM transactions
WHERE merchant IN (
    SELECT merchant
    FROM transactions
    WHERE status = 'Fraud'
);
```



Output:

- Customers using fraud-linked merchants

customer_name	merchant
Amit Sharma	Amazon
Neha Verma	Flipkart
Ravi Kumar	Amazon
Arjun Mehta	Apple Store
Amit Sharma	Amazon
Karan Patel	Flipkart
Vikas Rao	Amazon
Neha Verma	Flipkart
Rahul Jain	Samsung
Isha Desai	Apple Store
Amit Sharma	Amazon
Manish Gupta	Flipkart



Insight:

Some merchants are repeatedly involved in fraud cases.

8. City-wise Fraud Concentration

♦ SQL Query:

```
SELECT city, COUNT(*) AS total_transactions  
FROM transactions  
GROUP BY city;
```

Output:

- Transaction distribution by city

city	total_transactions
Pune	6
Mumbai	4
Delhi	3
Bangalore	1
Kolkata	1

Insight:

Some cities show higher transaction activity → higher risk zones.

9. Non-Fraud Customers

◆ SQL Query:

```
SELECT customer_name
FROM transactions
WHERE customer_id NOT IN (
  SELECT customer_id
  FROM transactions
  WHERE status = 'Fraud'
);
```

Output:

- Safe customers list

customer_name
Neha Verma
Pooja Singh
Sneha Iyer
Neha Verma
Isha Desai
Manish Gupta
Tanya Roy

Insight:

Helps in separating normal vs risky users.



10. Duplicate Transaction Pattern

◆ SQL Query:

```
SELECT amount, COUNT(*)  
FROM transactions  
GROUP BY amount  
HAVING COUNT(*) > 1;
```



Output:

- Repeated transaction amounts

amount	COUNT(*)
7000	2
200000	2



Insight:

Repeated identical amounts can indicate suspicious automated behavior.



11. Conclusion

This project demonstrates how SQL transforms raw transaction data into meaningful fraud insights.

✓ Key Takeaways:

- SQL helps detect anomalies in financial systems
- Fraud detection can be done using simple subqueries
- Aggregation reveals hidden patterns
- Data analysis is powerful even without AI tools